

*****Validation Control,**

- ➔ Validation is an important process to give valid input data with proper format from user to web form then give meaning full output.
- ➔ As we know all web forms must be validated properly.
- ➔ In ASP.Net we can validate form manually by doing coding or use readymade validation control from toolbox.

Types of Validation

There are two types of validation in ASP.Net

- 1.Client-Side Validation
2. Server-Side Validation

1.Client-Side Validation : Client side validation is done on the client browser. Client side validation is faster than sever side validation.

We use JavaScript, JQuery for client side validation.

2. Server-Side Validation : Server side validation is done on the web server. We use custom logic code for server side validation.

ASP.Net provides the following validation controls :

ASP.Net Validation controls are server side as well client side.

1.CompareValidator Control :

- > CompareValidator control is used to compare the value of an input control against the value of another input control on basis of specified operator.
- > CompareValidator control compare inputted value with the another input control value.

Syntax

```
<asp:CompareValidator ID="CompareValidator1" runat="server" ErrorMessage="CompareValidator">
</asp:CompareValidator>
```

List of Properties of CompareValidator control.

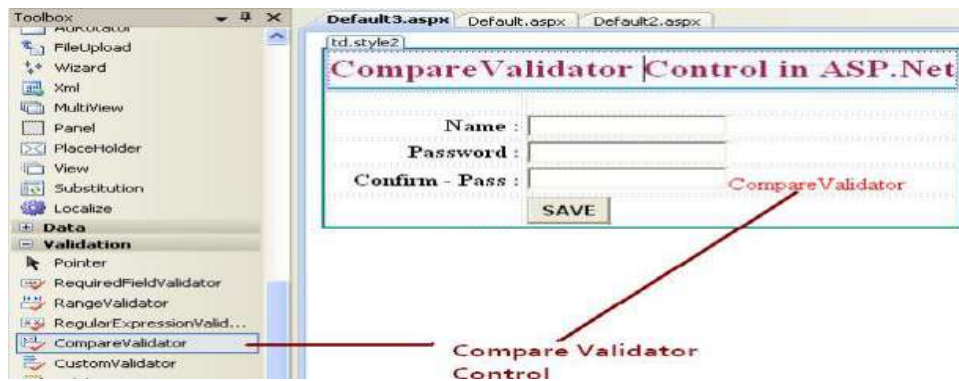
Properties	Description
ErrorMessage	Message to display in a ValidationSummary when validation fails.
Text	Text to display when validation fails.
ConrolToCompare	Set ID of input control to compare with.
ControlToValidate	Set ID of input control to validate.
Operator	Set comparison operator. (Equal, NotEqual, LessThan, GreaterThan etc.)
Type	It is used to define Data type of values for comparison.

Example

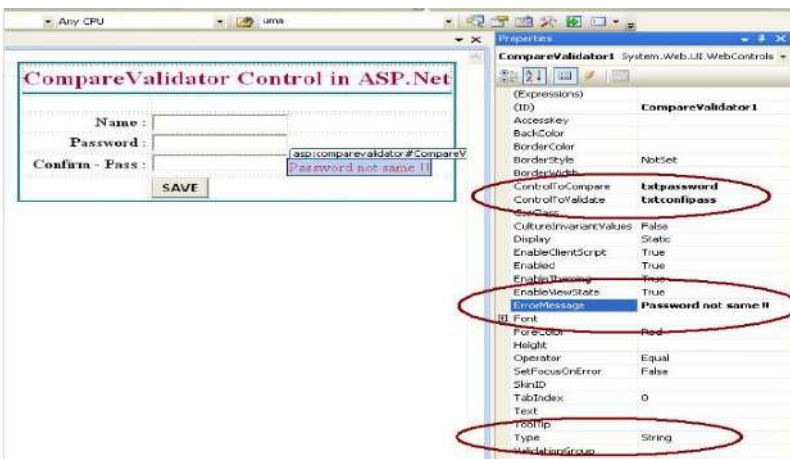
To creating registration form with password textbox. Here, we have three textbox control for name, password and compare password. Both password and compare password textbox value must be same.

HTML Code:

```
Name : <asp:TextBox ID="txtname" runat="server"></asp:TextBox>
Password : <asp:TextBox ID="txtpassword" runat="server"></asp:TextBox>
Confirm - Pass : <asp:TextBox ID="txtconfipass" runat="server"></asp:TextBox>
<asp:CompareValidator ID="CompareValidator1" runat="server" ControlToCompare="txtpassword"
ControlToValidate="txtconfipass" ErrorMessage="Password not same !!"></asp:CompareValidator>
```



- ➔ **ControlToValidate** – The main input control means Original first **password textbox**.
- ➔ **ControlToCompare** – The second input control which is compare with main control, means **confirm -password textbox** control.
- ➔ **Type** – Set data type String
- ➔ **Operator** – Equal



2.RangeValidatator Control

- >The RangeValidator control is used to check the input control value is within a specified range or not.
- > In other words we can say the input values must be between two defined values. It has minimum and maximum value.
- > RangeValidator control mostly used for a validate **Mobile No** and **Pincode**.
- > Mobile no and Pincode no length must be pre defined.
- > In India, Mobile no length must be 10 digits and Pincode must be 6 digits.

Syntax :

```
<asp:RangeValidator ID="RangeValidator1" runat="server" ErrorMessage="RangeValidator"></asp:RangeValidat
or>
```

List of Properties of RangeValidator control.

Properties	Description
ControlToValidate	Set ID of input control to validate.
ErrorMessage	Message to display in a ValidationSummary when validation fails.
MaximumValue	It is used to define Maximum Value.
MimimumValue	It is used to define Minimum Value.
Type	It is used to define Data type of values for comparison.

Example

Design a webform like below screen. Here we took three textbox control for name, mobile and pincode. We use RangeValidator for mobile number and poncode number. Mobile number length must be 10 digits and pincode number length must be 6 digits.

Mobile :

```
<asp:TextBox ID="txtmobile" runat="server"></asp:TextBox>
<asp:RangeValidator ID="RangeValidator1" runat="server"
ControlToValidate="txtmobile" ErrorMessage="Invalid Mobile !!" MaximumValue="9999999999"
```

```
MinimumValue="1111111111" Type="Double"></asp:RangeValidator>
```

Pincode :</td>

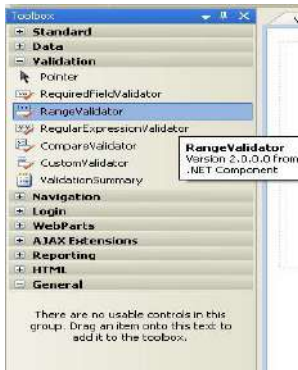
```
<asp:TextBox ID="txtpincode" runat="server"></asp:TextBox>
```

```
<asp:RangeValidator ID="RangeValidator2" runat="server"
```

```
ControlToValidate="txtpincode" ErrorMessage="Invalid Pincode !!"
```

```
MaximumValue="999999" MinimumValue="111111" Type="Integer"></asp:RangeValidator>
```

Now, Drag and drop RangeValidator control on web forms like below screen.



We must set three most important properties of RangeValidator.

MaximumValue –Specify the Maximum Value of Input Value.

MinimumValue – Specify the Minimum Value of Input Value.

Type – Specify the Data Type of Input Value.

For Validate Mobile number (10 digits) :

MinimumValue = 1111111111

MaximumValue = 9999999999

Type = Double

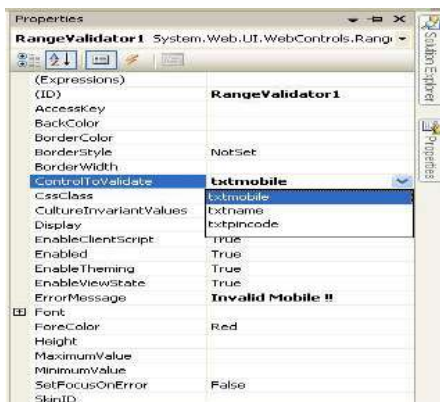
For Validate Pincode Number (6 digits) :

MinimumValue = 111111

MaximumValue = 999999

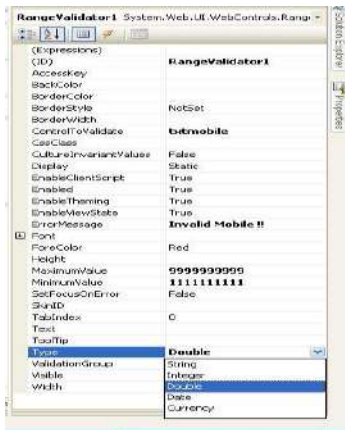
Type = Double

Here, assign ID of mobile number textbox to ControlToValidate property as shows in below screen.



How to use RangeValidator control in asp.net

Set MaximumValue and MinimumValue Property of RangeValidator control for validate mobile number.



3.RegularExpressionValidator Control

->RegularExpressionValidator control is used to validate the input value of control with the pattern of a regular expression.

->RegularExpressionValidator control mostly used for a validate **Internet E-mail address** and **Internet URL**.

->The Internet Email address must contain @ symbol and dot(.) in between some text.

> Same way in Internet URL means internet website name or domain name must start with http://.

Syntax :

```
<asp:RegularExpressionValidator ID="RegularExpressionValidator1" runat="server" ErrorMessage="RegularExpressionValidator"></asp:RegularExpressionValidator>
```

List of Properties of RegularExpressionValidator control.

Properties	Description
ControlToValidate	Set ID of input control to validate.
ErrorMessage	Message to display in a ValidationSummary when validation fails.
Text	Text to display when validation fails.
Validation Expression	It is used to set regular expression to check validity with control value.

Example

Design asp.net web forms with three textbox control along with a button control. After that drag and drop one RequiredFieldValidator control for name textbox and two RegularExpresionValidator control for validate both email and website url textbox.

Here, we make first textbox mandatory, user must have to input some value in name text box and rest of two must be validated properly with email and website url.



RegularExpressionValidator control we must have to set below property :

ControlToValidate = TextBox control which will be validate.

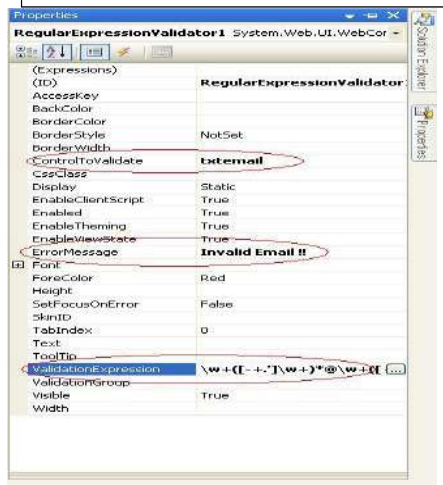
Text = Display error message.

Validation Expression = Select the expression for validation criteria.

Set the **Validation Expression property** of RegularExpressionValidator control. Select Internet e-mail address option for validate email address.



RegularExpression Validation in asp.net Set ControlToValidate, Text and ValidationExpression property like below screen:



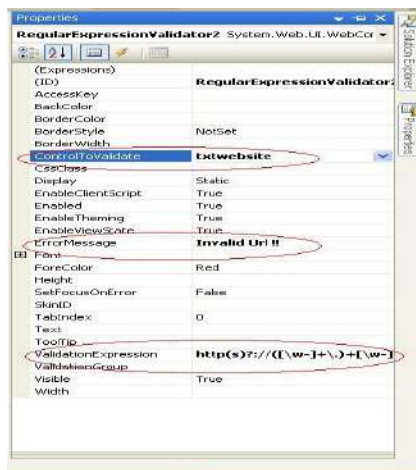
RegularExpression Validation in asp.net

If you want to validate Internet website URL then just select **Internet URL** option from Validation expressions. Select Internet URL option for validate website URL.



RegularExpression Validation in asp.net

If we select Internet URL option from expression then the property windows look like below screen.



RegularExpression Validation in asp.net

Output 1:

RegularExpression Validator in ASP.Net

Name:

Email: Invalid Email !!

Website: Invalid Url !!

RegularExpression Validation in asp.net

Output 2:

RegularExpression Validator in ASP.Net

Name:

Email:

Website:

RegularExpression Validation in asp.net

4.RequiredFieldValidator Control

->RequiredFieldValidator control is used to check that the validated control (textbox) must contain a value.
 ->If validated textbox control empty, RequiredFieldValidator control shows error message as we mention in text property in validator control_

Syntax :

```
<asp:RequiredFieldValidator ID="RequiredFieldValidator1" runat="server" ErrorMessage="RequiredFieldValidator"></asp:RequiredFieldValidator>
```

List of Properties of RequiredFieldValidator control.

Properties	Description
ControlToValidate	Set ID of input control to validate.
Display	Indicates how to error message will display.
ErrorMessage	Message to display in a ValidationSummary when validation fails.
InitialValue	Indicates the data inputted in a control is equal to or not to initial value.
Text	Text to display when validation fails.

Example

Name :

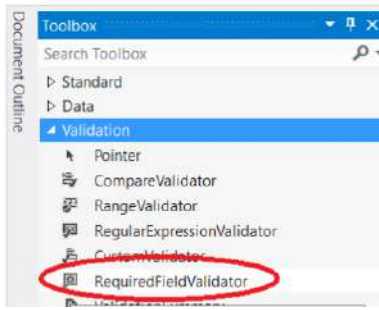

```

<asp:TextBox ID="txtname" runat="server" ></asp:TextBox>
<asp:RequiredFieldValidator ID="RequiredFieldValidator1" runat="server" ControlToValidate="txtname"
ErrorMessage="RequiredFieldValidator" ForeColor="Red">
Enter Name !!</asp:RequiredFieldValidator>
City :
<asp:TextBox ID="txtcity" runat="server"></asp:TextBox>
<asp:RequiredFieldValidator ID="RequiredFieldValidator2" runat="server" ControlToValidate="txtcity"
ErrorMessage="RequiredFieldValidator" ForeColor="Red">
Enter City !!</asp:RequiredFieldValidator>

```

Example in ASP.Net

Let's understand RequiredFieldValidator Control Example to creating mandatory TextBox controls. Design asp.net web forms with two textbox control along with a button control. After that drag and drop two RequiredFieldValidator control for validate both textbox. Here, we make both textbox mandatory, user must have to input some value in both textbox control if you leave blank textbox validation control shows the error message. Drag and drop validation control from toolbox as shown in below figure.



Here, we define two textbox one for name and other for city. we have two RequiredFieldValidator control one for name textbox and other for city textbox.

Here, we assign txtname as name textbox id and txtcity as city textbox id.

RequiredFieldValidator1

Set ControlToValidate = (txtname) id of first textbox.

Set Text = "EnterName !!" for error message.

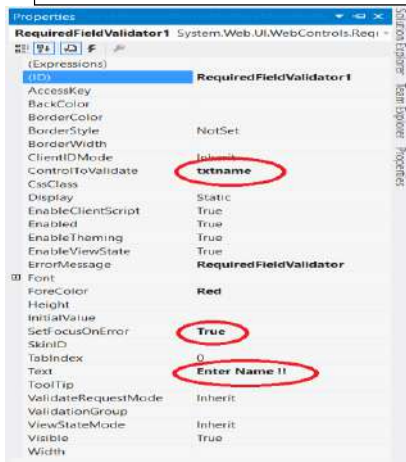
Set SetFocusOnError = True

RequiredFieldValidator2

Set ControlToValidate = (txtcity) id of second textbox.

Set Text = "EnterName !!" for error message.

Set SetFocusOnError = True



5.ValidationSummary Control

->The ValidationSummary control is used to display a summary of all the validation control error messages in a single control on web forms in asp.net.

->In other word we can say the validationSummary control is a summary of all the validation control error messages display on web forms.

-> ValidationSummary control allow user to display summary of all validation errors at single place.

Syntax

```
<asp:ValidationSummary ID="ValidationSummary1" runat="server" />
```

List of Properties of ValidationSummary control.

Properties	Description
DisplayMode	Format for error summary display. (List, BulletList, SingleParagraph)
ShowMessageBox	True / False – display summary error message box like javascript alert.
ShowSummary	True / False – display summary error message box on webpage.

Example

Design the asp.net web forms with four textbox and four validation control and a button control along with the ValidationSummary control.

Name :<asp:TextBox ID="txtname" runat="server"></asp:TextBox>

<asp:RequiredFieldValidator ID="RequiredFieldValidator1" runat="server" ControlToValidate="txtname" ErrorMessage="Enter Name !!"></asp:RequiredFieldValidator>

City :<asp:TextBox ID="txtcity" runat="server"></asp:TextBox>

<asp:RequiredFieldValidator ID="RequiredFieldValidator2" runat="server" ControlToValidate="txtcity" ErrorMessage="Enter City !!"></asp:RequiredFieldValidator>

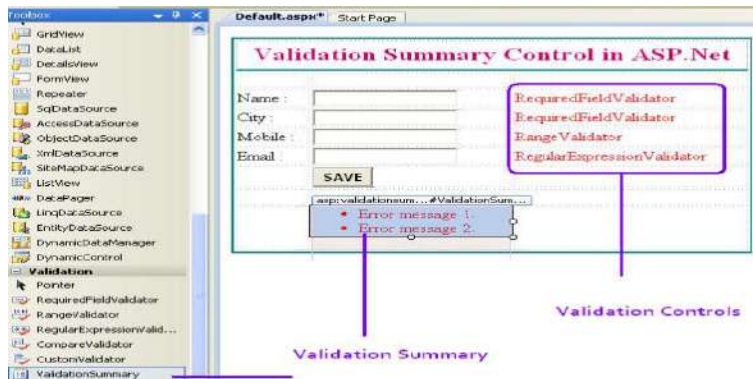
Mobile :</td>

<asp:TextBox ID="txtmobile" runat="server"></asp:TextBox>

```
<asp:RangeValidator ID="RangeValidator1" runat="server" ControlToValidate="txtmobile" ErrorMessage="Invalid Mobile no !!"MaximumValue="9999999999" MinimumValue="1000000000" Type="Double"></asp:RangeValidator>
```

```
<asp:ValidationSummary ID="ValidationSummary1" runat="server" DisplayMode="List" ShowMessageBox="True" />
```

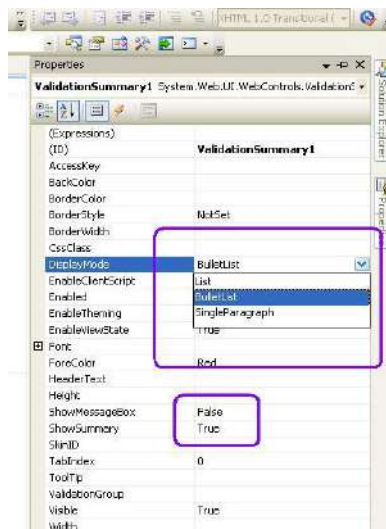
Here is the design of web forms :



There are three method or layout for display validationSummary control on asp.net web forms. If you need to change the layout of validationSummary then set the **DisplayMode** property of control.

DisplayMode = List, BulletList, SingleParagraph

Below, figure shows how to change or set DisplayMode property of validationSummary control in asp.net.



If we set **DisplayMode=BulletList** then the validationsummary example result look like a below screen. All the error messages are together listed as bullet list.

Validation Summary Control in ASP.Net

Name : Enter Name !!

City : Enter City !!

Mobile : Invalid Mobile no !!

Email : Invalid Email Address !!

DisplayMode = BulletList

Validation Summary

- Enter Name !!
- Enter City !!

How to use ValidationSummary Control in ASP.Net

The Result for **DisplayMode = SingleParagraph**, all the error messages are displayed as single paragraph format.

Validation Summary Control in ASP.Net

Name : Enter Name !!

City : Enter City !!

Mobile : Invalid Mobile no !!

Email : Invalid Email Address !!

Enter Name !! Enter City !! Invalid Mobile no !! Invalid Email Address !!

DisplayMode = SingleParagraph

How to use ValidationSummary Control in ASP.Net

If we set **DisplayMode=List**, then validationSummary output will be like this.

Validation Summary Control in ASP.Net

Name : Enter Name !!

City : Enter City !!

Mobile : Invalid Mobile no !!

Email : Invalid Email Address !!

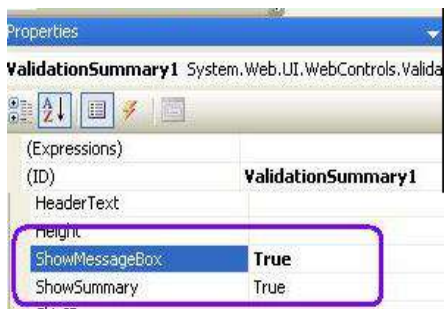
Enter Name !!
Enter City !!
Invalid Mobile no !!
Invalid Email Address !!

DisplayMode = List

How to use ValidationSummary Control in ASP.Net

There are another two properties of validationSummary control we need to understand the first one is **ShowMessageBox** and **ShowSummary**.

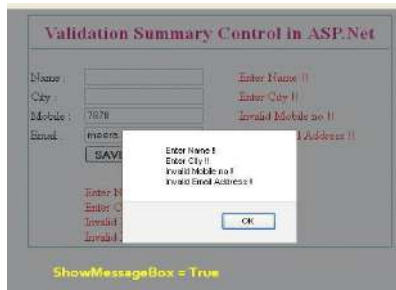
If **ShowMessageBox = True**. So the ValidationSummary control error message display as javascript alert box.



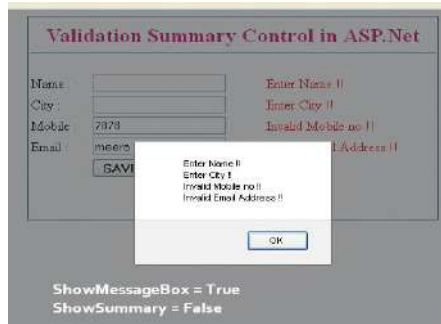
Here, we set the **ShowMessageBox= True**, we can see here all the validation messages in alert box.

How to use ValidationSummary Control in ASP.Net

The ShowSummary property used to show or hide the validationSummary error messages on web forms.



Here, we can set **ShowMessageBox=True** or **ShowSummary=False**



How to use ValidationSummary Control in ASP.Net

Navigation Control

Basically ASP.NET 2.0 has three navigation controls:

1. Dynamic menus
2. Tree Views
3. Site Map Path

(1) Dynamic menus

It was the very difficult task to maintain the menu of a large website and time consuming. It is used to display the Menus. You can use it as easy as other Navigation controls. Menu can be stored in a file to make it easier to maintain. This file is normally called web. Sitemap, and is stored in the root directory of the web.

Example:

```

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title>Untitled Page</title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Menu ID="Menu1" runat="server">
        <DynamicItemTemplate>
          Audio songs
          <br />
          Video songs
          <br />
        </DynamicItemTemplate>
        <StaticItemTemplate>
          Songs
          <br />
        </StaticItemTemplate>
        <Items>
          <asp:MenuItem Text="Songs" Value="Songs">
            <asp:MenuItem Text="New Item" Value="New Item"></asp:MenuItem>
          </asp:MenuItem>
        </Items>
      </asp:Menu>
    </div>
  </form>
</body>
</html>

```

(2) Tree Views

A Tree View control displays a hierarchical list of items using lines to connect related items in a hierarchy. Each item consists of a label and an optional bitmap. Windows Explorer uses a Tree View control to display directories. You can use the Tree View control in any situation in which you need to display hierarchical data.

Source code of Tree View control is as follows:

```
<asp:TreeView ID="TreeView1" runat="server"> </asp:TreeView>
```

Example: We want to create the Employee's information in the hierarchical structure by using the Tree View control.

```

<%@ Page Language="C#" AutoEventWireup="true" CodeFile="TreeView.aspx.cs" Inherits="sapna_Navigation_Controls_TreeView" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title>Untitled Page</title>
</head>

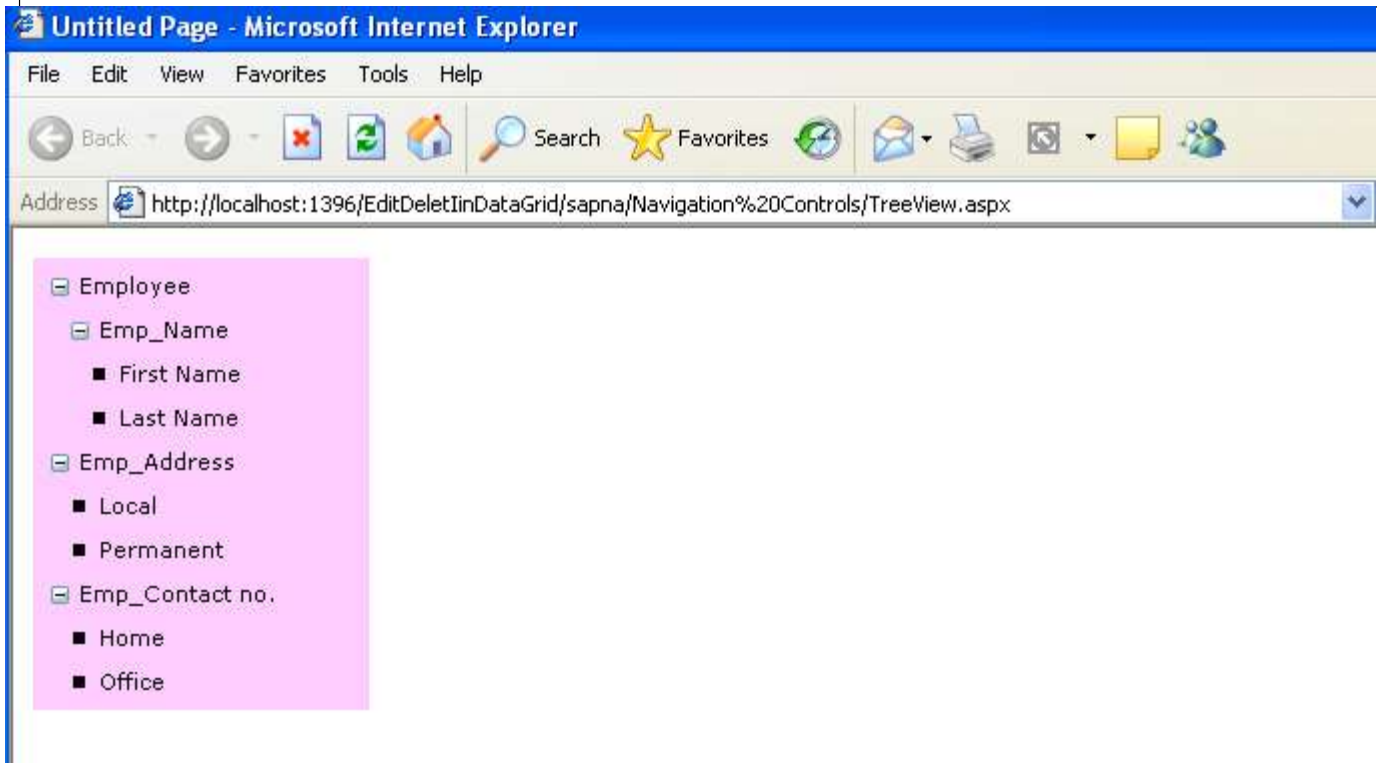
```

```

<body>
  <form id="form1" runat="server">
    <div>
      <table bgcolor="#ffccff">
        <tr>
          <td>
            <asp:TreeView ID="TreeView1" runat="server" Height="189px" ImageSet="Simple" Width
            ="162px"
              NodeIndent="10">
                <ParentNodeStyle Font-Bold="False" />
                <HoverNodeStyle Font-Underline="True" ForeColor="#DD5555" />
                <SelectedNodeStyle Font-
                Underline="True" ForeColor="#DD5555" HorizontalPadding="0px"
                VerticalPadding="0px" />
                <Nodes>
                  <asp:TreeNode Text="Employee" Value="Employee">
                    <asp:TreeNode Text="Emp_Name" Value="Emp_Name">
                      <asp:TreeNode Text="First Name" Value="First Name"></asp:TreeNode>
                      <asp:TreeNode Text="Last Name" Value="Last Name"></asp:TreeNode>
                    </asp:TreeNode>
                  </asp:TreeNode>
                  <asp:TreeNode Text="Emp_Address" Value="Emp_Address">
                    <asp:TreeNode Text="Local" Value="Local"></asp:TreeNode>
                    <asp:TreeNode Text="Permanent" Value="Permanent"></asp:TreeNode>
                  </asp:TreeNode>
                  <asp:TreeNode Text="Emp_Contact no." Value="Emp_Contact no.">
                    <asp:TreeNode Text="Home" Value="Home"></asp:TreeNode>
                    <asp:TreeNode Text="Office" Value="Office"></asp:TreeNode>
                  </asp:TreeNode>
                </Nodes>
                <NodeStyle Font-Names="Verdana" Font-
                Size="8pt" ForeColor="Black" HorizontalPadding="0px"
                NodeSpacing="0px" VerticalPadding="0px" />
              </asp:TreeView>
            </td>
          </tr>
        </table>
      </div>
    </form>
  </body>
</html>

```

Output of the above source code is as follows:



(3) Site Map Path

Use of this control is very simple. You can add this control to your page then view your page in browser. The Sitemap Path control displays the navigation path of the current page. The path acts as click able links to previous pages.

The Sitemap Path control uses the web. Sitemap file by default.

Code of Sitemap Path:

```
<asp: SiteMapPath ID="SiteMapPath1" runat="server"></asp: SiteMapPath>
```

Sitemap Path representation:

Example: In this Example you will see the source code of the Sitemap Path with some properties.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="SiteMapPath.aspx.cs" Inherits="sapna_Navi
gation_Controls_SiteMapPath" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0
Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title>Untitled Page</title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
```

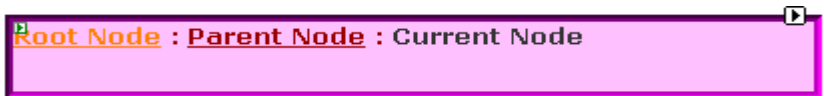


```

<asp:SiteMapPath ID="SiteMapPath1" runat="server" Font-Names="Verdana" Font-Size="0.8em"
  PathSeparator=" : " BackColor="#FFC0FF" BorderColor="Fuchsia" BorderStyle="Inset"
  Height="40px" Width="400px">
  <PathSeparatorStyle Font-Bold="True" ForeColor="#990000" />
  <CurrentNodeStyle ForeColor="#333333" />
  <NodeStyle Font-Bold="True" ForeColor="#990000" />
  <RootNodeStyle Font-Bold="True" ForeColor="#FF8000" />
</asp:SiteMapPath>
</div>
</form>
</body>
</html>

```

Design view of the Site map Path:



, Web Control Event

ASP.NET provides important feature event handling to Web Forms. It let us to implement event-based model for our application. As a simple example, we can add a button to an ASP.NET Web Forms page and then write an event handler for the button's click event. ASP.NET Web Forms allows events on both client and server sides.

In ASP.NET Web Forms pages, however, events associated with server controls originate on the client but are handled on the Web server by the ASP.NET.

Example: Creating Event Handler

Here, we are creating an event handler for click event. In this example, when user click on the button, an event fires and handler code executes at server side.

// EventHandler.aspx

```

using System.Web.UI.WebControls;
namespace asp.netexample
{
    public partial class EnventHandling : System.Web.UI.Page

```

```

{
    protected void Button1_Click(object sender, EventArgs e)
    {
        int a = Convert.ToInt32(firstvalue.Text);
        int b = Convert.ToInt32(secondvalue.Text);
        total.Text = (a + b).ToString();
    }
}

```

Output:

The screenshot shows a web browser window with the address bar displaying 'http://localhost:51070/EnventH'. The browser's address bar also shows 'localhost:51070/EnventH' and a search bar. The page content includes three text input fields. The first field is labeled 'First value' and contains the number '12'. The second field is labeled 'Second value' and also contains '12'. The third field is labeled 'Sum' and contains '24'. Below these fields is a blue button labeled 'Sum' with a mouse cursor hovering over it.

Web Control– Method –

In ASP.NET, web controls are server-side controls that are used to create interactive user interface elements on a web page. These controls include elements like buttons, text boxes, labels, and more. They are rendered as HTML on the client side, but they offer server-side functionalities, which means you can manage user interactions using C# code on the server.

Example of a Web Control: Button

Theory

A `Button` web control in ASP.NET is used to submit a form or trigger a server-side event when clicked by the user. It has an `OnClick` event handler, which is a method that executes when the button is clicked. This is typically where you would put your business logic to handle the click event.

Simple Example

ASP.NET Web Form (.aspx file):

```

<html
<head runat="server">
  <title>Button Web Control Example</title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Label ID="Label1" runat="server" Text="Click the button:"></asp:Label>
      <br /><br />
      <asp:Button ID="Button1" runat="server" Text="Click Me!" OnClick="Button1_Click" />
      <br /><br />
      <asp:Label ID="Label2" runat="server" Text=""></asp:Label>
    </div>
  </form>
</body>
</html>

```

Code-behind file (Default.aspx.cs):

using System;

```

public partial class Default : System.Web.UI.Page
{
    protected void Page_Load(object sender, EventArgs e)
    {
    }

    protected void Button1_Click(object sender, EventArgs e)
    {
        Label2.Text = "Button clicked at " + DateTime.Now.ToString();
    }
}

```

- **ASP.NET Web Form (.aspx file):**

- The <asp:Button> control is a server-side button.
- The OnClick attribute specifies the event handler (Button1_Click) in the code-behind file.
- The <asp:Label> controls are used to display text on the web page.

- **Code-behind (Default.aspx.cs):**

- Button1_Click is the event handler method that gets triggered when the button is clicked.
- Inside this method, the text of Label2 is set to the current date and time, demonstrating a simple interaction.

Web Control– Property,

In ASP.NET, web controls come with a variety of properties that allow you to configure their behavior, appearance, and functionality. Properties can be set either declaratively in the `.aspx` file or programmatically in the code-behind file.

Example of a Web Control Property: `TextBox`*Theory*

A `TextBox` control in ASP.NET is used to accept user input. It has several properties, one of the most common being the `Text` property, which holds the value entered by the user.

- **Text Property:** The `Text` property of a `TextBox` control is used to get or set the text contained in the text box.

Simple Example

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default.aspx.cs" Inherits="Default" %>
<!DOCTYPE html>
<html >
<head runat="server">
  <title>TextBox Property Example</title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Label ID="Label1" runat="server" Text="Enter your name:"></asp:Label>
      <br /><br />
      <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
      <br /><br />
      <asp:Button ID="Button1" runat="server" Text="Submit" OnClick="Button1_Click" />
      <br /><br />
      <asp:Label ID="Label2" runat="server" Text=""></asp:Label>
    </div>
  </form>
</body>
</html>
```

Code-behind file (`Default.aspx.cs`):
using System;

```
public partial class Default : System.Web.UI.Page
{
  protected void Page_Load(object sender, EventArgs e)
  {
  }

  protected void Button1_Click(object sender, EventArgs e)
  {
    // Accessing the Text property of the TextBox control
    string name = TextBox1.Text;
```

```
// Setting the Text property of the Label control
Label2.Text = "Hello, " + name + "!";
}
}
```

- **ASP.NET Web Form (.aspx file):**

- The <asp:TextBox> control is a server-side control where users can enter text.
- The ID attribute of the TextBox is TextBox1, which is used to reference this control in the code-behind file.
- The <asp:Button> control triggers the Button1_Click event when clicked.
- The <asp:Label> control (Label2) is used to display a greeting message based on the user's input.

- **Code-behind (Default.aspx.cs):**

- The Text property of the TextBox1 control is accessed to retrieve the user's input.
- The Text property of the Label2 control is set to display a greeting message, incorporating the text entered by the user.

When you run this example, the user can enter their name into the TextBox and click the Submit button. The Button1_Click method retrieves the name using the Text property of TextBox1 and then sets the Text property of Label2 to display a personalized greeting.

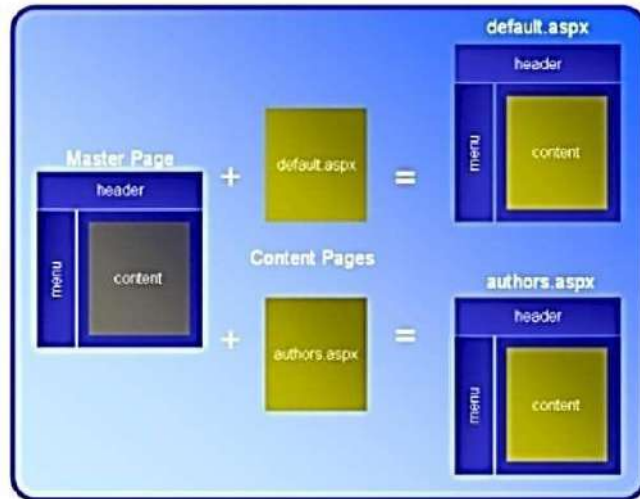
Master Page,

- ➔ ASP.NET master pages allow you to create a consistent layout for the pages in your application.
- ➔ A single master page defines the look and feel and standard behavior that you want for all of the pages (or a group of pages) in your application.
- ➔ the master page is a feature that enables you to define common structure and interface markup elements for your Web site, including headers, footers, style definitions, or navigation bars.
- ➔ Master pages actually consist of two pieces, the master page itself and one or more content pages
- ➔ .A master page is an ASP.NET file with the extension .

```
<%@ Master Language="C#" CodeFile="MasterPage.master.cs" Inherits="MasterPage" %>
```

A master page provides a template for other pages, with shared layout and functionality. The master page defines placeholders for the content, which can be overridden by content pages. The output result is a combination of the master page and the content page.

The content pages contain the content you want to display.



When users request the content page, ASP.NET merges the pages to produce output that combines the layout of the master page with the content of the content page.

Use of Master Pages

The master pages can be used to accomplish the following:

- Creating a set of controls that are common across all the web pages and attaching them to all the web pages.
- A centralized way to change the above created set of controls which will effectively change all the web pages.
- Dynamically changing the common UI elements on master page from content pages based on user preferences.

Advantages of Master Pages

Master pages provide functionality that developers have traditionally created by copying existing code, text, and control elements repeatedly; using framesets; using include files for common elements; using ASP.NET user controls; and so on. Advantages of master pages include the following:

- They allow you to centralize the common functionality of your pages so that you can make updates in just one place.
- They make it easy to create one set of controls and code and apply the results to a set of pages. For example, you can use controls on the master page to create a menu that applies to all pages.
- They give you fine-grained control over the layout of the final page by allowing you to control how the placeholder controls are rendered.
- They provide an object model that allows you to customize the master page from individual content pages.

Terminology

- **Masterpage:** Gives us a way to create common set of UI elements that are required on multiple pages of our website.
- **ContentPage:** The ASP.NET web page that will use master page to have the common UI elements displayed on rendering itself.
- **ContentPlaceHolder:** A control that should be added on the MasterPage which will reserve the area for the content pages to render their contents.
- **ContentControl:** A control which will be added on content pages to tell these pages that the contents inside this control will be rendered where the MasterPage's ContentPlaceHolder is located.

Using ASP.NET's Master Page technology for your web application development will drastically reduce the amount of code required for each page by eliminating duplicated interface elements and the supporting code.

In its simplest form, master pages are simply user controls. When a .NET server receives a page request, it simply inserts the master page content into the requested content page (not the other way around).

Content pages normally don't have header, sidebar and footer content because that usually comes from the Master page. To search engines or users, your page URLs look like any other .NET page URL. Only the server knows it's inserting a master page control.

Be aware, there may be some slight performance issues due to the server having to recreate the master page control on every postback. Using output caching, checking for postbacks and using other performance techniques, you should be able to eliminate bottlenecks common with web applications using Master pages.

Theme.

A theme is a collection of property settings that allow you to define the look of pages and controls, and then apply the look consistently across pages in a Web application, across an entire Web application, or across all Web applications on a server.

Themes and Control Skins

Themes are made up of a set of elements: skins, cascading style sheets (CSS), images, and other resources. At a minimum, a theme will contain skins. Themes are defined in special directories in your Web site or on your Web server.

Custom made themes are saved inside the predefined "*App_Themes*" folder inside ASP.NET applications, making them easier to manage and use according to your needs. The essential part of themes are skin files with the *.skin* extension. Besides skin files, a theme can be composed of styles sheet *.css* files as well as images for added support for the layout of the website.

Skins

A skin file has the file name extension .skin and contains property settings for individual controls such as Button, Label, TextBox, or Calendar controls. Control skin settings are like the control markup itself, but contain only the properties you want to set as part of the theme. For example, the following is a control skin for a Button control:

```
<asp:button runat="server" BackColor="lightblue" ForeColor="black" />
```

You create .skin files in the Theme folder. A .skin file can contain one or more control skins for one or more control types. You can define skins in a separate file for each control or define all the skins for a theme in a single file.

There are two types of control skins, default skins and named skins:

- A default skin automatically applies to all controls of the same type when a theme is applied to a page. A control skin is a default skin if it does not have a SkinID attribute. For example, if you create a default skin for a Calendar control, the control skin applies to all Calendar controls on pages that use the theme. (Default skins are matched exactly by control type, so that a Button control skin applies to all Button controls, but not to LinkButton controls or to controls that derive from the Button object.)
- A named skin is a control skin with a SkinID property set. Named skins do not automatically apply to controls by type. Instead, you explicitly apply a named skin to a control by setting the control's SkinID property. Creating named skins allows you to set different skins for different instances of the same control in an application.

Cascading Style Sheets

A theme can also include a cascading style sheet (.css file). When you put a .css file in the theme folder, the style sheet is applied automatically as part of the theme. You define a style sheet using the file name extension .css in the theme folder.

Theme Graphics and Other Resources

Themes can also include graphics and other resources, such as script files or sound files. For example, part of your page theme might include a skin for a TreeView control. As part of the theme, you can include the graphics used to represent the expand button and the collapse button.

Typically, the resource files for the theme are in the same folder as the skin files for that theme, but they can be elsewhere in the Web application, in a subfolder of the theme folder for example. To refer to a resource file in a subfolder of the theme folder, use a path like the one shown in this Image control skin:

```
<asp:Image runat="server" ImageUrl="ThemeSubfolder/filename.ext" />
```

You can also store your resource files outside the theme folder. If you use the tilde (~) syntax to refer to the resource files, the Web application will automatically find the images. For example, if you place the resources for a theme in a subfolder of your application, you can use paths of the form ~/SubFolder/filename.ext to refer to resource files, as in the following example.


```
<asp:Image runat="server" ImageUrl="~/AppSubfolder/filename.ext" />
```

There are two types of themes:

- **Customization themes:** These types of themes are applied after the properties of the control are applied, meaning that the properties of the themes override the properties of the control itself.
- **Stylesheet themes:** You can apply this type of theme to a page in exactly the same manner as a customization theme. However, stylesheet themes don't override control properties, thus allowing the control to use the theme properties or override them.

Themes vs. Cascading Style Sheets

Themes are similar to cascading style sheets in that both themes and style sheets define a set of common attributes that can be applied to any page. However, themes differ from style sheets in the following ways:

- Themes can define many properties of a control or page, not just style properties. For example, using themes, you can specify the graphics for a TreeView control, the template layout of a GridView control, and so on.
- Themes can include graphics.
- Themes do not cascade the way style sheets do. By default, any property values defined in a theme referenced by a page's Theme property override the property values

declaratively set on a control, unless you explicitly apply the theme using the StyleSheetTheme property. For more information, see the Theme Settings Precedence section above.

- Only one theme can be applied to each page. You cannot apply multiple themes to a page, unlike style sheets where multiple style sheets can be applied.

Uses of Themes

1. Since themes can contain CSS files, images and skins, you can change colors, fonts, positioning and images simply by applying the desired themes.
2. You can have as many themes as you want and you can switch between them by setting a single attribute in the web.config file or an individual aspx page. Also you can switch between themes programmatically.
3. Setting the themes programmatically, you are offering your users a quick and easy way to change the page to their likings.
4. Themes allow you to improve the usability of your site by giving users with vision problems the option to select a high contrast theme with a large font size.

Data Controls –

DataList,

-> This control can display fields of various data sources in a custom format (not necessarily using Table labels).

-> The format of the displayed data is defined in the created template, you can create templates for items, alternate items, selected items, and edit items.

-> The datalist control can also use the title, footer, and delimiter template to customize the overall appearance, and display multiple data rows in one row.

-> Although the datalist control has great flexibility, it does not support data paging. programmers must compile their own methods to complete the paging function.

Syntax:

```
<asp:DataList ID="DataList1" runat="server">
```

```
</asp:DataList>
```

Example

```
protected void Page_Load(object sender, EventArgs e)
{
    DataTable table = new DataTable();
    table.Columns.Add("ID");
    table.Columns.Add("Name");
    table.Columns.Add("Email");
    table.Rows.Add("101", "Sachin Kumar", "sachin@example.com");
    table.Rows.Add("102", "Peter", "peter@example.com");
    table.Rows.Add("103", "Ravi Kumar", "ravi@example.com");
    table.Rows.Add("104", "Irfan", "irfan@example.com");
}
```

```

DataList1.DataSource = table;
DataList1.DataBind();
}

```

OUTPUT

The DataList shows data of DataTable

ID: 101 Name: Sachin Kumar Email: sachin@example.com
ID: 102 Name: Peter Email: peter@example.com
ID: 103 Name: Ravi Kumar Email: ravi@example.com

DetailView,

-> The detailsview control displays only one record of the data source at a time in the form of a table tag, and each row under the table tag (TR tag) represents a field in the record.

-> This control also supports data editing, insertion, and deletion, and allows you to easily set the paging function.

-> However, the detailsview control does not support data sorting

Syntax:

```
<asp:DetailsView ID="DetailsView1" runat="server" Height="50px"
```

```
Width="125px">
```

```
</asp:DetailsView>
```

Example

```
private void BindDetailsView()

{
    SqlConnection SQLConn = new SqlConnection("Data
    Source=COMPUTER_1\\SQLEXPRESS;Initial Catalog=BOOK;Integrated
    Security=True");
    SqlDataAdapter SQLAdapter = new SqlDataAdapter("select * from UserMst",
    SQLConn);
    DataTable DT = new DataTable();
    SQLAdapter.Fill(DT);
    DetailsView1.DataSource = DT;
    DetailsView1.DataBind();
}
```

OUTPUT

DetailsView Control in ASP.Net C#	
Name :	
City :	
SAVE	
ID	1
Name	Meera
City	Modasa

FormView,

- >The formview control only displays a single record in the data source.
- >Different from the detailsview control, the detailsview control uses the table layout (Table label). Each field in the record is displayed as a row.
- >The formview control is not used to display the preset layout of records.
- >Programmers need to create subitem templates and compile various controls used to display fields in records and other HTML labels for layout.
- >Like the detailsview control, the formview control can easily enable the paging function. If only one record is displayed, the formview control is recommended because you can customize the data display format while developing it efficiently.

Syntax:

```
<asp:FormView ID="FormView1" runat="server">  
    </asp:FormView>
```

Example

```
private void BindFormView()  
{  
    string CS = ConfigurationManager.ConnectionStrings["DBCS"].ConnectionString;  
    using (SqlConnection con = new SqlConnection(CS))  
    {  
        SqlCommand cmd = new SqlCommand("spGetStudentResult", con);  
        cmd.CommandType = CommandType.StoredProcedure;  
        SqlDataAdapter sda = new SqlDataAdapter(cmd);
```

```

con.Open();
DataTable dt = new DataTable();
sda.Fill(dt);
FormView1.DataSource = dt;
FormView1.DataBind();
}

```

OUTPUT

Displaying Data with the FormView

EmpID:	6
Name:	Digvijay
Gender:	Male
Salary:	45000
Address:	New York
DepID:	3
Edit New Delete	
1 2 3 4 5 6 7 8	

GridView,

- > This control can display, edit, and delete data in a variety of different data sources (such as databases, XML files, and collections) in the form of tables.
- > The gridview control is very powerful. If needed, programmers do not need to write any Code , Drag and drop through vs 2008, set properties from the property panel, and complete functions such as paging, sorting, and appearance settings.
- > Although the functions are very complete Program Performance will be affected.
- > It is best not to use this control too much on the page. Of course, if you need to display various data in a custom format, the gridview control also provides the template function for editing the format.

Syntax:

```
<asp:GridView ID="GridView1" runat="server">  
    </asp:GridView>
```

Example

```
private void BindGrid()  
{  
    string constr = ConfigurationManager.ConnectionStrings["constr"].ConnectionString;  
    string query = "SELECT * FROM Customers";  
    using (SqlConnection con = new SqlConnection(constr))  
    {  
        using (SqlDataAdapter sda = new SqlDataAdapter(query, con))  
        {  
            using (DataTable dt = new DataTable())  
            {  
                sda.Fill(dt);  
                GridView1.DataSource = dt;  
                GridView1.DataBind();  
            }  
        }  
    }  
}
```

OUTPUT

stor_id	stor_name	stor_address	city	state	zip		
6380	Eric the Read Books11	788 Catamangus Ave.	Seattle_11	WA	98056	Edit	Delete
7066	Barnum's	567 Pasadena Ave.	Tustin	CA	92789	Edit	Delete
7067	News & Brews	577 First St.	Los Gatos	CA	96745	Edit	Delete
7131	Doc-U-Mat: Quality	24-A Avogadro Way	Remulade	WA	98014	Edit	Delete
7896	Fricative Bookshop	89 Madison St.	Fremont	CA	90019	Edit	Delete
8042	Bookbeat	679 Carson St.	Portland	OR	89076	Edit	Delete
						Add New Row	

ListView,

- >The listview control displays data in the template format written by the programmer.
- >The listview control applies to any data with duplicate structures.
- >The listview control provides data operations such as editing, inserting, and deleting data. -> It also provides the data sorting and paging functions, which can be directly set in vs 2008, you do not need to write code, which is very similar to the gridview control.
- >It can be said that listview has both open templates of the Repeater control and editing features of the gridview control.

Syntax:

```
<asp:ListView ID="ListView1" runat="server">
    </asp:ListView>
```

Example

```
private void BindListView()
```



```
{
string constr = ConfigurationManager.ConnectionStrings["constr"].ConnectionString;
using (SqlConnection con = new SqlConnection(constr))
{
    using (SqlCommand cmd = new SqlCommand())
    {
        cmd.CommandText = "SELECT CustomerId, ContactName, Country FROM
Customers";
        cmd.Connection = con;
        using (SqlDataAdapter sda = new SqlDataAdapter(cmd))
        {
            DataTable dt = new DataTable();
            sda.Fill(dt);
            ListView1.DataSource = dt;
            ListView1.DataBind();
        }
    }
}
}
```

OUTPUT

EmpID	EmpName	Department	Age	Address
1	Satheesh	IT	25	Bangalore
3	Ramesh	CSE	23	Chennai
4	Arun	ECE	25	Coimbatore
5	Vijay	MEC	26	Coimbatore
6	Ram	IT	26	Coimbatore

Repeater

-> The Repeater control is a data binding container control, used to generate a list of each sub-item, the Display Method of these subitems can be completely edited by the programmer.

-> When the control is running on the page, the control displays the data format defined in the template based on the number of data rows in the data source.

-> Programmers can fully grasp the data display layout, such as Div and ul.

Note: The Repeater control only provides the duplicate template Content Function and does not provide paging, sorting, editing, and other functions. These functions must be implemented by programmers themselves.

Syntax:

```
<asp:Repeater ID="Repeater1" runat="server">
    </asp:Repeater>
```

Example

```
public partial class _Default : System.Web.UI.Page
{
```

```
SqlConnection con;  
SqlCommand cmd = new SqlCommand();  
protected void Page_Load(object sender, EventArgs e)  
{  
    con = new SqlConnection(ConfigurationManager.ConnectionStrings["constr"].Conne  
ctionString);  
    cmd.Connection = con;  
    cmd.CommandText = "select * from student";  
    con.Open();  
    Repeater1.DataSource = cmd.ExecuteReader();  
    Repeater1.DataBind();  
    con.Close();  
}  
}
```

Output

Roll No	Student Name	Total Fees
1	Puran	1200
2	Reena	3400
3	Patrick	3400
4	Rahul	4500
5	Sanjay	2000
8	Mahesh	5600
9	Sundar	2300
8	Poonam	6700
9	Yogesh	2300
10	Vinod	9000
6	York	1200
7	Manish	4500
School Records displayed		

Extra Most IMP Exam Related Questions

The Code-Behind Page Model

The code-behind page model allows you to keep the markup in one file—the .aspx file—and the programming code in another file. The name of the code file varies according to what programming language you are using.

In the code-behind model, the example used in the preceding section for the single-file page would be in two parts. The markup would be in one file (in this example, SamplePage.aspx) and would be similar to the single-file page, as shown in the following code example.

```
<%@ Page Language="C#" CodeFile="SamplePage.aspx.cs"
```

```

Inherits="SamplePage" AutoEventWireup="true" %>
<html>
<head runat="server" >
  <title>Code-Behind Page Model</title>
</head>
<body>
  <form id="form1" runat="server">
    <div>
      <asp:Label id="Label1" runat="server" Text="Label" >
    </asp:Label>
    <br />
    <asp:Button id="Button1"
      runat="server"
      onclick="Button1_Click"
      Text="Button" >
    </asp:Button>
    </div>
  </form>
</body>
</html>

```

The code-behind file contains the complete class declarations in the default namespace. However, the class is declared with the partial keyword, which indicates that the class is not contained entirely in one file. Instead, the compiler reads the .aspx page and the file it references in the @Page directive, and assembles them into a single class. The partial class file inherits from the page Page class.

```

using System;
using System.Web;
using System.Web.UI;
using System.Web.UI.WebControls;
public partial class SamplePage : System.Web.UI.Page
{
    protected void Button1_Click(object sender, EventArgs e)
    {
        Label1.Text = "Clicked at " + DateTime.Now.ToString();
    }
}

```

Choosing a Page Model

The single-file and code-behind page models are functionally the same. At run time, the models execute the same way, and there is no performance difference between them. Choosing a page model therefore depends on other factors, such as how you want to organize the code in your application, whether it is important to separate page design from coding, and so on.

Advantages of Single-File Pages

As a rule, the single-file model is suitable for pages in which the code consists primarily of event handlers for the controls on the page.

Advantages of the single-file page model include the following:

- In pages where there is not very much code, the convenience of keeping the code and markup in the same file can outweigh other advantages of the code-behind model. For example, it can be easier to study a single-file page because you can see the code and the markup in one place.
- Pages written using the single-file model are slightly easier to deploy or to send to another programmer because there is only one file.
- Because there is no dependency between files, a single-file page is easier to rename if you are using tools other than Visual Studio. (If you use Visual Studio to rename a file, Visual Studio automatically renames both files.)
- Managing files in a source code control system is slightly easier, because the page is self-contained in a single file.

Advantages of Code-Behind Pages

Code-behind pages offer advantages that make them suitable for Web applications with significant code or in which multiple developers are creating a Web site.

Advantages of the code-behind model include the following:

- Code-behind pages offer a clean separation of the markup (user interface) and code. It is practical to have a designer working on the markup while a programmer writes code.
- Code is not exposed to page designers or others who are working only with the page markup.
- Code can be reused for multiple pages.

To make the coding style user-friendly and keeping the old style ASP.NET provides two types of coding model.

Inline Code / Single page (Classical Model)

The coding style used by ASP developers is known as inline coding, inline code or in-page code because that was the only way to develop an ASP page.

In ASP Pages scripting code working for generating desired output was intermixed with HTML code to create user-friendly pages as well as specific functionality was added to the web pages.

But with the latest ASP.NET introduction the old developers are moving from the old classical style of programming to the new style.

Code-Behind Model

The ASP.NET Framework has provided the way out to maintain the code for large web pages. Now you can design the HTML code page with .aspx extension separately and maintain the code files for the same .aspx page of .aspx.cs and .aspx.vb separately. This style of coding to develop web pages is called Code-Behind.

Code Behind approach is a better way to develop and design the .aspx page having basic layout of a web page containing all the necessary controls required for the GUI of the web page. Then include the C# or VB code behind class file for handling the events of controls. This mechanism separates the web page from design layout from the coding part.

Difference between Inline code and Code-behind.

Inline / Single	Code Behind
The business logic is in <script> blocks in the same .aspx file that contains the HTML and controls.	The HTML and controls are in the .aspx file, and the business logic is in a separate .aspx.cs or .aspx.vb file.
When the page is deployed, the sourcecode is deployed along with the Web Forms page, since it is physically in the .aspx file. Though, we are not able to see the code, only the results are rendered when the page runs.	All project class files (without the .aspx file) are compiled into a .dll file, which are deployed to the server without any source code. When a request for the page is received, then an instance of the project .dll file is created and executed.
The .aspx file derives from the Page class.	The code for the page is compiled into a separate class from which the .aspx file derives.
When we write inline code we write code in the same page with Html code between scripting tags. So Each time when there is a request for page it compiles the code each time then server the page Like classic asp because inline code cannot create dll.	The code-behind approach also improved productivity (at some level) since the designer and the developer can continue working simultaneously on the same set of application. It's also easier to build & test the UI and the business logic (DLL) - separately or combined.